From libwpe to WPEPlatform: A new way to render embedded WebKit

Building an Embedded Browser on Raspberry Pi with the New WPE API

Web Engines Hackfest

June 2026, A Coruña

Kyungsun (Kate) Lee

Igalian since 2025 · South Korea

- WebKit committer since 2026
- Implements WPE WebKit & new web-platform features
- Worked on the Samsung TV browser
- Android & Linux-based app development

GitHub: **@kate-k-lee** · Blog: **blogs.igalia.com/klee**

igalia

Outline

I

What is WPE WebKit?

The embedded WebKit port — and why it exists

II

The libwpe model — and where it breaks

The legacy stack and the problems integrators hit

III

Enter WPEPlatform

What the new API is, and why it's better

IV

On the Raspberry Pi 4

Building a Yocto image, and the performance picture

igalia

What is WPE WebKit?

The embedded WebKit port — and why it exists

igalia

WPE WebKit

WebKit's lightweight port for embedded — forked from WebKitGTK and released by Igalia in 2017

MAJOR BROWSER ENGINES

Chromium
(Blink)
Chrome, Edge…

WebKit
Safari · WPE

Gecko
Firefox

Servo
research engine

WEBKIT'S PORTS

Apple	WebKitGTK	WPE	WinCairo	PlayStation
macOS · iOS (Safari)	Linux desktop (GNOME Web)	embedded · lightweight	Windows	consoles

- WebKit is one of the major browser engines .
- It isn't monolithic — it ships as several ports .
- WPE is the lightweight port, built for embedded devices.

igalia

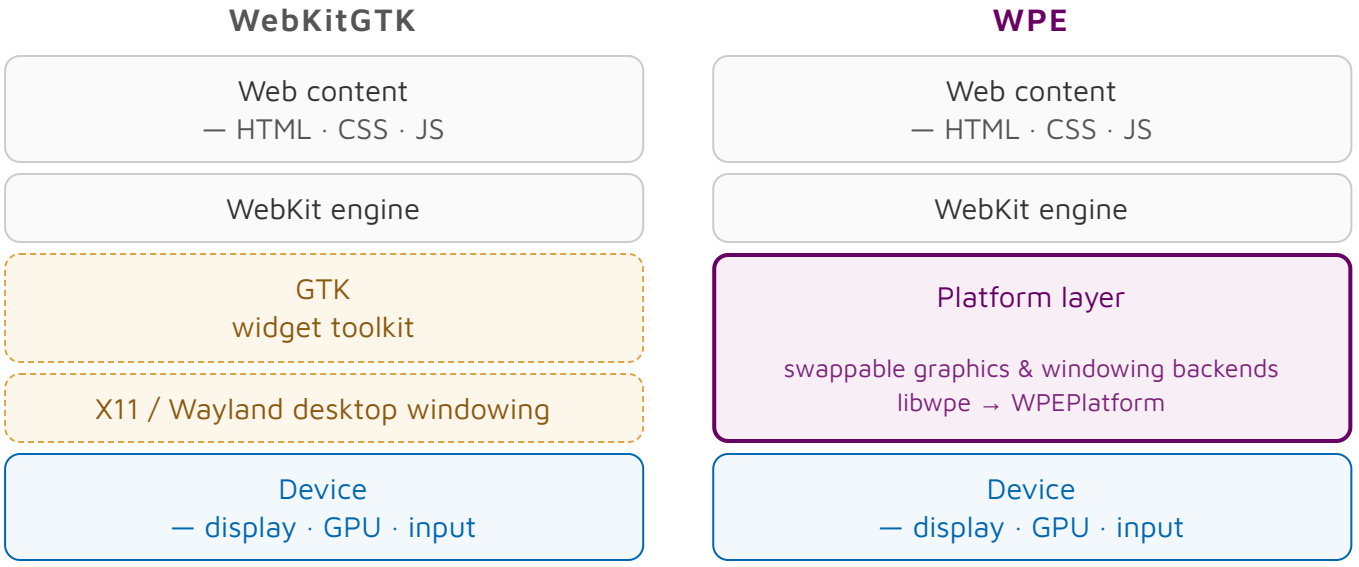
Why WPE exists

- Before WPE, WebKit ports were each **bound to a UI toolkit** such as **GTK**, **EFL** (Tizen / TVs), or **Qt** (automotive).
- That coupling was the problem: if the toolkit didn't fit your device, you had to swap the **whole port**.
- Igalia **forked WebKitGTK and stripped GTK out**, creating a port tied to **no toolkit** where you simply swap the **backend**.

igalia

WPE WebKit's key characteristics

- **No UI-toolkit dependency** — as we saw, WPE isn't tied to GTK or any toolkit.
- **A platform layer takes its place** — it abstracts how WebKit reaches the device through **easily swappable backends** (graphics, windowing), so the engine is **built & optimized for your device**, with no compositor or desktop assumed.



That platform layer is the whole story of this talk — it used to be libwpe, and it is becoming WPEPlatform.

Sources: **wpewebkit.org** · **Igalia — "Happy birthday WPE!"**

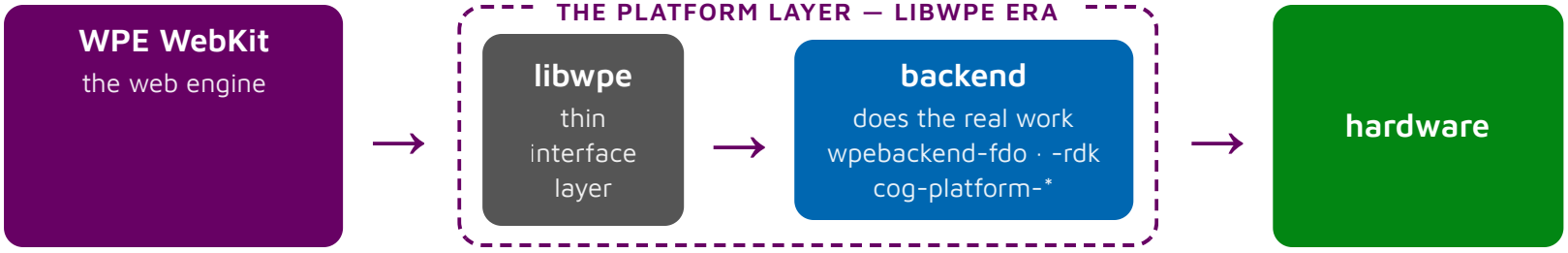
igalia

The libwpe model

The legacy stack — and where it breaks

igalia

libwpe: the legacy rendering stack



The rendering path — a frame travels from the web engine, through a backend, to the screen.

- **`libwpe` is intentionally tiny** — just a C **function-pointer interface** (`wpe_view_backend_*`, `wpe_renderer_*`). A thin interface layer with **no rendering logic of its own**.
- **The backends do everything** — each one wires WebKit to the device's **display, GPU and input**.
  e.g. `wpebackend-fdo` (reference — Wayland/EGL) · `wpebackend-rdk` (set-top boxes) · `cog-platform-*` (Cog's per-environment modules)
- *But the reference backend `wpebackend-fdo` requires a nested Wayland compositor — extra overhead on a constrained device.*

Sources: WPEBackend-fdo · WPEBackend-rdk · Cog (platform modules)

igalia

libwpe: where it breaks

Nested Wayland compositor

fdo runs its own compositor — extra overhead on a constrained device.

Deprecated Mesa extension

Relies on the deprecated `EGL_WL_bind_wayland_display` — a shrinking foundation.

Rigid C function pointers

Hand-wired function-pointer tables — verbose and error-prone.

Scattered across 3 projects

2 libraries (libwpe + wpebackend-fdo) + the Cog launcher — hard to evolve.

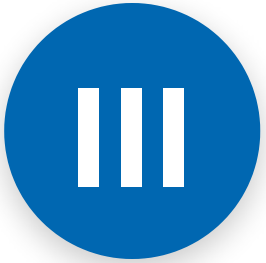
A libwpe backend is a hand-filled vtable of `void*` callbacks:

```
struct wpe_view_backend_interface {
    void* (*create)(void*, struct wpe_view_backend*);
    void  (*destroy)(void*);
    void  (*initialize)(void*);
    int   (*get_renderer_host_fd)(void*);

    /*< private >*/
    void (*_wpe_reserved0)(void);   // empty slots reserved to avoid ABI breaks
    void (*_wpe_reserved1)(void);
    void (*_wpe_reserved2)(void);
    void (*_wpe_reserved3)(void);
};
```

All four are solved by WPEPlatform — the new in-tree API we turn to next.

igalia



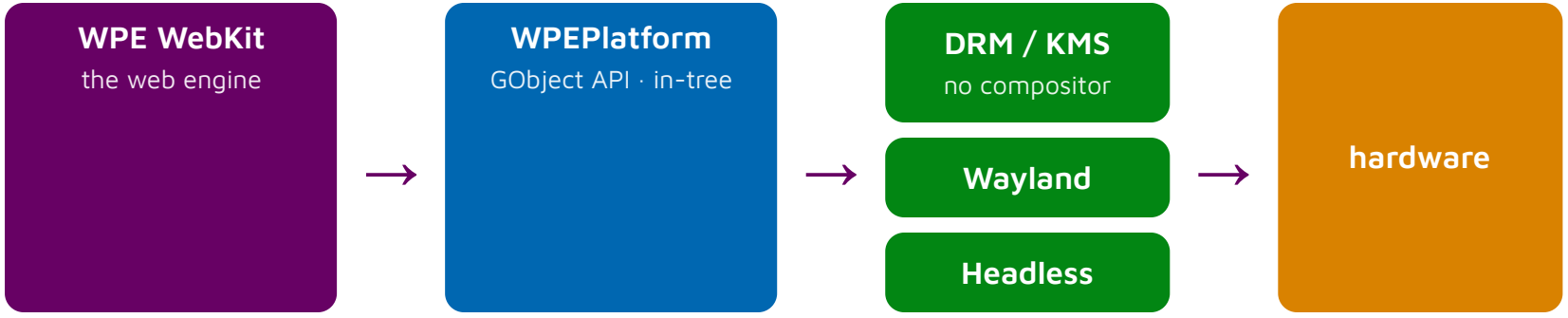
Enter WPEPlatform

What the new API is — and why it's better

igalia

WPEPlatform Architecture

GObject-based platform layer shipped **in-tree** with WPE WebKit — it **replaces the whole** `libwpe` **+** `wpebackend-*` **stack** with one coherent API.



- **Automatic backend** — the right one is created and chosen at runtime (`WPE_DISPLAY=…`, or auto-detected); no manual wiring
- **DMA-BUF direct sharing** — drops the deprecated `EGL_WL_bind_wayland_display`, and enables **direct scanout** (zero-copy to the panel)
- **GObject signals & introspection** — replace C function-pointer tables; bindings in any language. In my launcher this removed ~**250 lines of boilerplate**.

igalia

WPEPlatform by example — same task, both APIs

libwpe (legacy) — create an external backend, then wrap it:

```c
#include <wpe/wpe.h>
#include <wpebackend-fdo/fdo.h>

struct wpe_view_backend *backend = wpe_view_backend_create ();
wpe_fdo_initialize_for_egl_display (eglDisplay);          // external backend

WebKitWebViewBackend *vb = webkit_web_view_backend_new (backend, NULL, NULL);
WebKitWebView *webView   = webkit_web_view_new (vb);
```

WPEPlatform (new) — the backend is built in:

```c
#include <wpe/webkit.h>

WPEDisplay    *display = wpe_display_get_default ();        // env-selected backend
WebKitWebView *webView = WEBKIT_WEB_VIEW (g_object_new (WEBKIT_TYPE_WEB_VIEW, NULL));

WPEView *view = webkit_web_view_get_wpe_view (webView);
wpe_toplevel_maximize (wpe_view_get_toplevel (view));
```

On the Raspberry Pi 4

Building a Yocto image, running it, and the performance picture

igalia

Building a Yocto image for RPi 4

- **Base image** `core-image-weston-wpe` — `machine` **raspberrypi4-64-mesa** · `distro` **poky-wayland**; bundles **wpewebkit** + the **wpe-simple-launcher**
- Built with **kas** over the `meta-wpe-image` layer (Igalia) — every layer pinned → **reproducible**. Recipes: https://github.com/Igalia/meta-wpe-image
- **Pick the API at build time** (`wpewebkit` PACKAGECONFIG): `wpe-platform` → **WPEPlatform** · `wpe-legacy-api` → **libwpe** · both on → one `libWPEWebKit-2.0.so` exposes both

What WPE WebKit needs to build:

layer	source	role
poky (meta, meta-poky)	yoctoproject.org	Yocto / OE core + reference distro (`poky-wayland`)
meta-openembedded	openembedded.org	Mesa · Wayland · GStreamer · python · networking…
meta-raspberrypi	yoctoproject.org	RPi 4 BSP — kernel · bootloader · VC4
meta-webkit (Igalia)	github.com/Igalia	wpewebkit · libwpe · wpebackend-fdo — the core
meta-clang	github.com/kraj	builds WebKit with clang / libc++ on aarch64
meta-wpe-image (Igalia)	this layer	image · launcher · version-pin glue

igalia

Demo

The embedded browser on the Raspberry Pi 4, running on WPEPlatform.

Demo — on the device

WebGL Aquarium in WPE WebKit on the **Raspberry Pi 4** — hardware-accelerated WebGL through **WPEPlatform**, fullscreen on a small HDMI panel. The on-screen counter shows the live frame rate.

igalia

Full video: **https://cloud.igalia.com/s/jYWQbrdDYZGsTxC?dir=/&openfile=true**

igalia

Performance on the Raspberry Pi 4

MotionMark 1.3 @ 15 fps — overall rendering score on **RPi 4 (64-bit)**, tracked over time (higher = better).

The new API's **GPU** path stays consistently above **CPU** rendering and the **legacy libwpe** API.

Live dashboard: **https://wpe-perf-dashboard.igalia.com/**


igalia


igalia